# Tarea 1: Pseudocódigo

Fisica computacional

**Presentado por:** `William Andres Avilez Rios, Nicolas Palacios Cortez, y Breyner Lasso Arrieta`

**Resumen**

Este trabajo abordó conceptos fundamentales de programación, como el uso de variables, los tipos de datos y la construcción de algoritmos, empleando diagramas de flujo para representar procesos de forma gráfica. Se desarrollaron dos actividades principales, la elaboración de diagramas de flujo y pseudocódigos para tareas cotidianas (sacar un libro de la biblioteca, almorzar en central y preparar una tortilla), y la implementación de un programa en Python para calcular la altura máxima alcanzada por un proyectil dados unos parámetros conocidos.

Los diagramas de flujo facilitaron la descomposición de las tareas en pasos concretos y ordenados, mientras que el programa en Python permitió evidenciar la importancia de validar adecuadamente los datos de entrada como la velocidad inicial y el ángulo de lanzamiento para obtener resultados correctos.

## 1. Introducción

Este trabajo tiene como propósito analizar los conceptos esenciales de la programación, tales como las variables, los tipos de datos, las operaciones matemáticas y lógicas, y la estructura básica de los algoritmos, los cuales son fundamentales para almacenar, manipular y procesar información de forma eficiente. Asimismo, se tratará el uso de herramientas gráficas como los diagramas de flujo, que permiten representar visualmente la secuencia de pasos de un algoritmo, facilitando su diseño, implementación y depuración [1]. Comprender estos conceptos resulta clave para crear programas eficaces y funcionales. En este sentido, el trabajo pretende brindar una base sólida para desarrollar las competencias necesarias en el ámbito de la programación[2].

## 2. Problemas

**Problema 1: Diagramas de flujo**

**Introducción**
Desarrollar pseudocódigos y diagramas de flujo para diferentes problemas de la cotidianidad, con el fin de mostrar la relevancia de estos mismos frente a la programación y su utilidad a la hora de plantear los conductos lógicos de las soluciones implicadas.
**Metodología**
Se plantearon conductos lógicos no muy específicos, lo cual induce a una generación de vacíos los cuales pueden producir problemas, sin embargo, esta simplicidad redujo la cantidad de procesos, dando paso a la eficiencia.
**Resultados**
Los resultados obtenidos se pueden apreciar a continuación; pseudocódigo de como sacar un libro de la biblioteca departamental, hacer una tortilla y almorzar en cafetería central, y sus respectivos diagramas de flujo.

**Algorithm 1** Proceso para sacar un libro de la biblioteca

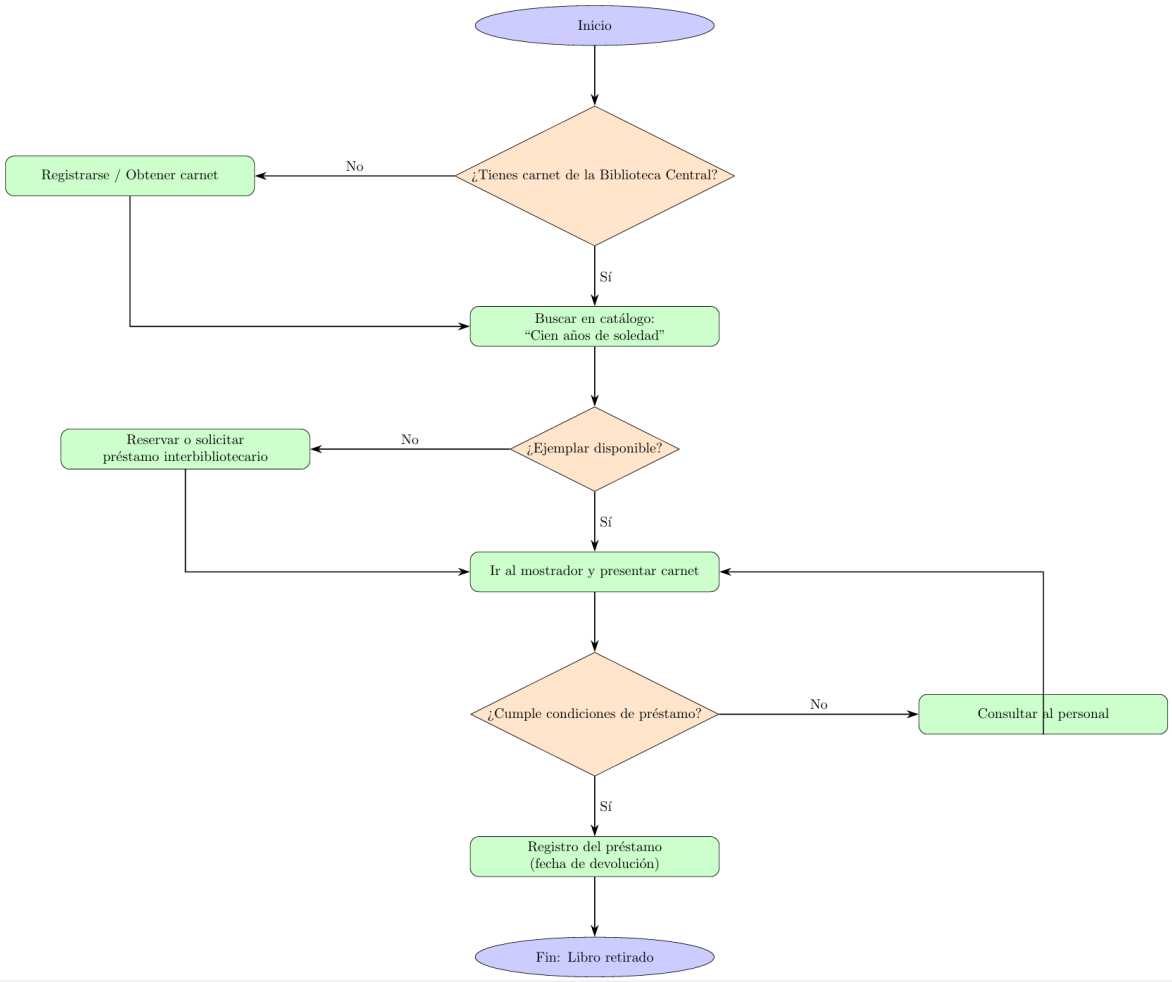
1: **Inicio**
2: ¿Tiene carnet de la biblioteca central? **if** *No tiene* **then**
- Obtener carnet, **else**
- Buscar en el catálogo "Cien años de soledad"

3: ¿Hay ejemplares disponibles? **if** *No hay* **then**
- Reservar o solicitar préstamo interbibliotecario, **else**
- Ir al mostrador y presentar carnet

4: ¿Cumple condiciones de préstamo? **if** *No cumple* **then**
- Consultar al personal, **else**
- Registrar el préstamo

5: **Fin**

Diagrama de flujo para sacar un libro de la biblioteca:



**Algorithm 2** Proceso para almorzar en central

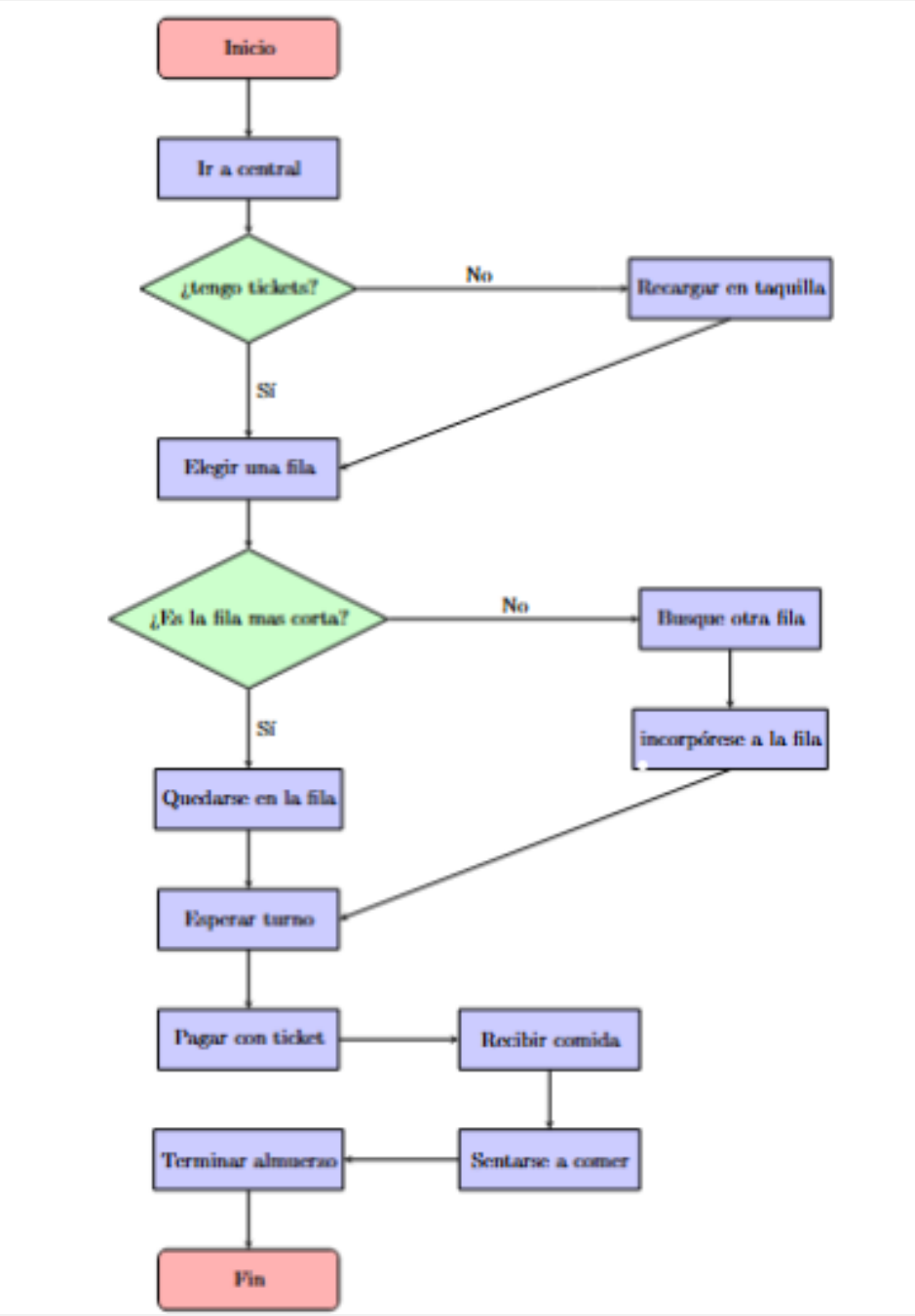
**Inicio**
Ir a central
¿Tiene tickets? **if** *No tiene* **then**
- Recargar en taquilla, **else**
- Elegir una fila

¿Es la fila no es la más corta? **if** *No* **then**
- Buscar otra fila

Incorporarse a la fila elegida, **else**
- Quedarse en la fila

Esperar turno
Pagar con ticket
Recibir comida
Sentarse a comer
Terminar el almuerzo
**Fin**

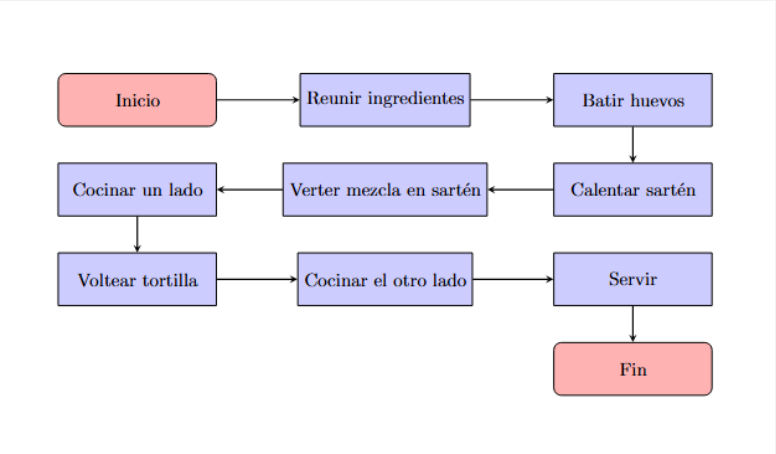
Diagrama de flujo para almorzar en central:



**Algorithm 3** Proceso para preparar de una tortilla

```
Inicio
Reunir todos los ingredientes necesarios
Batir los huevos hasta obtener una mezcla homogénea
Calentar la sartén a fuego medio
Verter la mezcla de huevos en la sartén caliente
Cocinar un lado de la tortilla hasta que cuaje
Voltear la tortilla con cuidado
Cocinar el otro lado hasta dorar ligeramente
Servir la tortilla en un plato
Fin
```

Diagrama de flujo para preparar una tortilla:




**Discusión**

La elaboración de pseudocódigos y diagramas de flujo para actividades cotidianas permitió evidenciar cómo el pensamiento algorítmico contribuye a estructurar acciones de forma lógica y eficiente. En el caso de sacar el libro de la biblioteca, se identificaron pasos clave como desplazarse, buscar el libro en el catálogo, localizarlo en la estantería y hacer el préstamo. Para almorzar en la Cafetería Central, fue necesario planificar desde el desplazamiento y elección de comida hasta el pago y consumo. Finalmente, para la receta de la tortilla, el proceso requirió establecer un orden estricto en la preparación de ingredientes y cocción.

Estos ejercicios demostraron que, aunque las tareas sean distintas, todas pueden descomponerse en pasos secuenciales que se prestan a ser formalizados como algoritmos.

**Conclusiones**

La realización de pseudocódigos y diagramas de flujo para tareas cotidianas permitió reforzar la comprensión de la lógica algorítmica y su aplicabilidad fuera del contexto de programación, y se comprobó que cualquier actividad puede ser descompuesta en pasos ordenados y representada gráficamente, lo cual favorece la planificación, la eficiencia y la detección de posibles errores.

**Problema 2**

**Introducción**

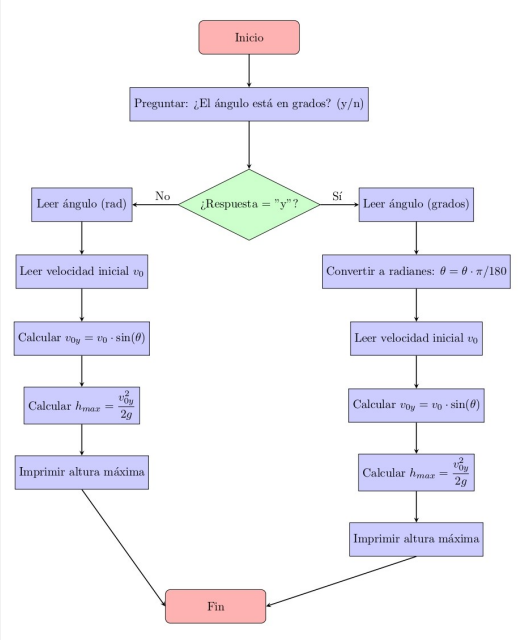
Dado un ángulo inicial y una velocidad, calcular la altura máxima alcanzada por el proyectil.

**Metodología**

Se propuso un código desarrollado en Python donde el usuario pueda tener la opción, no solo de usar un angulo y una velocidad fija, sino cualquiera que él desee. Luego, con uso de herramientas de cinemática, se describió como calcular la altura máxima alcanzada por un proyectil con las condiciones anteriormente registradas.

**Resultados**

Diagrama de flujo de calcular la altura máxima alcanzada por un proyectil:




Código desarrollado a partir del diagrama de flujo anterior.

```python
#Importamos las librerias necesarias
import numpy as np
import matplotlib.pyplot as plt
g=9.81
# El usuario define las varibles necesarias para describir la trayectoria.
# Puede ponerlo en grados o radianes, si el valor es el mismo no cambiara nada
respuesta=input("¿Al angulo esta dado en grados? (y/n): ")
if respuesta=="y":
    angulo=float(input("Ingrese el angulo teta (grados): "))
    angulo=angulo*np.pi/180
    v0=float(input("Ingrese la velocidad inicial (m/s): "))
    # hacemos el calculo de la altura maxima usando la cinematica.
    v0_y_ang=v0*np.sin(angulo)
    h_max=v0_y_ang**2/(2*g)
    print("La altura maxima es: ",h_max,"m")
# De igual manera para angulo representado en radianes:
else:
  radianes=float(input("Ingrese el angulo teta (rad): "))
  v0=float(input("Ingrese la velocidad inicial (m/s): "))
  v0_y_rad=float(v0*np.sin(radianes))
  h_max=v0_y_rad**2/(2*g)
  print("La altura maxima es: ",h_max,"m")
```

**Discusión**

El diagrama de flujo aporto considerablemente en el desarrollo del conducto lógico del código, sin embargo, fue muy especifico con las entradas admitidas por este mismo, ya que no acepta entradas de caracteres no numéricos (ej. "pi", "2pi","pi/2")

**Conclusiones**

Finalmente el diagrama de flujo permitió identificar los posibles problemas lógicos antes de iniciar la elaboración del código y facilito la búsqueda de soluciones implicadas

## Glosario

**Punto flotante**

Es una representación en memoria de números reales que permite manejar valores muy grandes o muy pequeños,.

**Programación**

Consiste en dar instrucciones a un computador mediante un lenguaje de programación para que realice tareas específicas, desde cálculos simples hasta sistemas complejos.

**Algoritmos**

Conjunto de pasos ordenados y finitos para resolver un problema.

**Cadenas de texto**

Es una secuencia ordenada de caracteres que se utiliza para representar palabras, frases u otro tipo de información textual dentro de un programa.

**Dispositivos de salida**

Son componentes de hardware que permiten a la computadora comunicar resultados procesados al usuario.

**Operaciones matemáticas y lógicas**

Son aquellas que permiten realizar cálculos numéricos básicos como suma, resta, multiplicación y división, así como operaciones más avanzadas y que permiten realizar cálculos numéricos básicos como suma, resta, multiplicación y división, así como operaciones más avanzadas.

**Pseudocódigo**

Representación intermedia de un algoritmo, escrita en lenguaje cercano al natural pero estructurado.

## 3. Conclusión

Esta tarea permitió integrar el uso de diagramas de flujo, pseudocódigo y programación en Python para abordar tanto procesos cotidianos como el cálculo de la altura máxima de un proyectil. La descomposición de cada actividad en pasos ordenados facilitó su comprensión y ejecución, mientras que el programa en Python demostró la relevancia de planificar y validar correctamente los datos de entrada para obtener resultados precisos; los resultados obtenidos confirmaron la utilidad de estas herramientas para diseñar algoritmos eficientes y aplicables en distintos contextos reales.

## Referencias


[1] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, Cambridge, UK, 3rd edition, 2007.

[2] Niklaus Wirth. Program development by stepwise refinement. *Communications of the ACM*, 14(4):221–227, 1971.


## 4. Anexos

1.Funcionamiento del código Python para valores en grados:

```python
#Importamos las librerias necesarias
import numpy as np
import matplotlib.pyplot as plt
g=9.81
# El usuario define las varibles necesarias para describir la trayectoria.
# Puede poner el angulo en grados o radianes, si el valor es el mismo no cambiara nada.
respuesta=input("¿Al angulo esta dado en grados? (y/n): ")
if respuesta=="y":
    angulo=float(input("Ingrese el angulo teta (grados): "))
    angulo=angulo*np.pi/180
    v0=float(input("Ingrese la velocidad inicial (m/s): "))
    # hacemos el calculo de la altura maxima usando la cinematica.
    v0_y_ang=v0*np.sin(angulo)
    h_max=v0_y_ang**2/(2*g)
    print("La altura maxima es: ",h_max,"m")
# De igual manera para angulo representado en radianes:
else:
  radianes=float(input("Ingrese el angulo teta (rad): "))
  v0=float(input("Ingrese la velocidad inicial (m/s): "))
  v0_y_rad=float(v0*np.sin(radianes))
  h_max=v0_y_rad**2/(2*g)
  print("La altura maxima es: ",h_max,"m")
```

```
¿Al angulo esta dado en grados? (y/n): y
Ingrese el angulo teta (grados): 45
Ingrese la velocidad inicial (m/s): 50
La altura maxima es:  63.71049949031597 5 m
```

2.Funcionamiento del código Python para valores en radianes:

```python
#Importamos las librerias necesarias
import numpy as np
import matplotlib.pyplot as plt
g=9.81
# El usuario define las varibles necesarias para describir la trayectoria.
# Puede poner el angulo en grados o radianes, si el valor es el mismo no cambiara nada.
respuesta=input("¿Al angulo esta dado en grados? (y/n): ")
if respuesta=="y":
    angulo=float(input("Ingrese el angulo teta (grados): "))
    angulo=angulo*np.pi/180
    v0=float(input("Ingrese la velocidad inicial (m/s): "))
    # hacemos el calculo de la altura maxima usando la cinematica.
    v0_y_ang=v0*np.sin(angulo)
    h_max=v0_y_ang**2/(2*g)
    print("La altura maxima es: ",h_max,"m")
# De igual manera para angulo representado en radianes:
else:
  radianes=float(input("Ingrese el angulo teta (rad): "))
  v0=float(input("Ingrese la velocidad inicial (m/s): "))
  v0_y_rad=float(v0*np.sin(radianes))
  h_max=v0_y_rad**2/(2*g)
  print("La altura maxima es: ",h_max,"m")
```

```
¿Al angulo esta dado en grados? (y/n): n
Ingrese el angulo teta (rad): 0.78
Ingrese la velocidad inicial (m/s): 50
La altura maxima es:  63.022673479977854 m
```